specified task, and then goes back to the queue for more work. In the simplest case there's no way to wait for the task to complete. If you need to do this, you have to manage the synchronization yourself.

The following listing shows a sample implementation of such a thread pool.

Listing 9.1. Simple thread pool

```
class thread_pool
{
    std::atomic_bool done;
    thread_safe_queue<std::function<void()> > work_queue;       <-1
    std::vector<std::thread> threads;                           <-2
    join_threads joiner;                                        <-3

    void worker_thread()
    {
        while(!done)                                            <-4
        {
            std::function<void()> task;
            if(work_queue.try_pop(task))                        <-5
            {
                task();                                         <-6
            }
            else
            {
                std::this_thread::yield();                      <-7
            }
        }
    }
```